

Interview Prep — DPI Engine & MeshPay

Read this like a script, but don't memorize it word for word — say it in your own voice. Both projects come from the same root interest: you like computer networks, and you wanted to actually *build* the packet-level machinery instead of just reading RFCs. Lead with that connection when you introduce them.

1. Deep Packet Inspection (DPI) Engine

30-second pitch (say this first)

"I built a DPI engine in Python that reads raw PCAP files, manually parses Ethernet/IPv4/TCP/UDP headers byte-by-byte — no Scapy, no libpcap bindings — identifies which application generated each flow by pulling the SNI out of the TLS ClientHello, the Host header from HTTP, or the query name from DNS, and then applies allow/block rules by IP, app, or domain. I built it two ways: a single-threaded version, and a multi-threaded version modeled on a real router's data plane — load balancers hashing packets to fast-path worker threads by 5-tuple, so I could reason about consistent hashing and per-flow ordering guarantees."

Why you built it (motivation, always ask this)

You wanted to understand what your ISP or a corporate firewall is actually doing when it says "we detected YouTube traffic" — since almost everything is now TLS-encrypted, deep packet inspection can't read payloads anymore, so it has to fingerprint traffic from the small amount of metadata that's still visible in plaintext during a TLS handshake (the SNI field) or in cleartext HTTP.

Step-by-step: what happens when you run it

```
python main.py input.pcap output.pcap --block-app YouTube
```

1. **PCAP file reading** (`pcap_reader.py`): The PCAP file format starts with a 24-byte global header (magic number, version, snaplen, link-layer type). I read the magic number first to detect byte order (little vs big-endian) and validate it's actually a PCAP file. Then I loop: read a 16-byte per-packet header (timestamp + captured length + original length), then read exactly that many bytes as the raw frame. This gives me a stream of raw Ethernet frames with timestamps.

2. **Packet parsing** (`packet_parser.py`): For each raw frame I manually unpack bytes using Python's `struct` module — no dpkt/Scapy:
 - Bytes 0–5 = destination MAC, 6–11 = source MAC, 12–13 = EtherType (0x0800 = IPv4).
 - Then the IPv4 header: version+IHL nibble (to get header length, since IPv4 headers are variable length due to options), TTL, protocol number (6=TCP, 17=UDP), source/dest IP.
 - Then TCP or UDP: source/dest port, sequence/ack numbers, TCP flags, and I compute the data offset to find where the header ends and the payload begins.
 - Everything after the transport header is `payload_data` — that's what gets fed to the fingerprinting stage.
3. **Flow tracking**: Each unique 5-tuple (src IP, dst IP, src port, dst port, protocol) is treated as a "flow" — a dataclass that accumulates packet/byte counts and remembers what app it belongs to once identified, so I don't have to re-run fingerprinting on every single packet of a connection.
4. **Traffic fingerprinting** — this is the core DPI logic, three techniques depending on destination port:
 - Port 443 (TLS) → SNI extraction (`sni_extractor.py`): I check if the payload looks like a TLS ClientHello (content type 0x16, a valid TLS version, handshake type 0x01), then manually walk the TLS record structure — skip the record header, handshake header, client random (32 bytes), session ID (variable length, length-prefixed), cipher suites list (length-prefixed), compression methods (length-prefixed) — until I reach the extensions block. I scan extensions for type 0x0000 (server_name), and inside that extension the hostname is length-prefixed UTF-8 bytes. That hostname is the SNI — e.g. `www.youtube.com` — visible in plaintext even over an encrypted connection, because the SNI extension isn't encrypted in TLS 1.2/1.3 (unless ECH is used).
 - Port 80 (HTTP) → Host header extraction: check if the payload starts with a known HTTP method (`GET`, `POST`, etc.), then find `Host:` in the payload and read until CRLF.
 - Port 53 → DNS query name: parse the DNS message — 12-byte header, then a sequence of length-prefixed labels (e.g. `[3]www[6]google[3]com[0]`) that I join with dots.
 - Whatever hostname I extract gets matched against a keyword table (`sni_to_app_type`) to classify it as YouTube, Google, Netflix, WhatsApp, etc.
5. **Rule enforcement** (`rule_manager.py`): Rules are three simple sets — blocked IPs, blocked app types, blocked domain substrings. `is_blocked()` checks all three against the flow. Once a flow is marked blocked, every subsequent packet in that flow is dropped without re-checking (efficiency).
6. **Output**: Forwarded packets get written to a new PCAP file (`pcap_writer`) with the same PCAP format; blocked packets are silently dropped. At the end I print a report:

total packets, TCP/UDP counts, per-app percentage breakdown with a little ASCII bar chart, and the list of detected domains.

The multi-threaded engine — the part interviewers dig into most

I modeled this on how a real hardware router's data plane is architected — separating control decisions from packet forwarding, and parallelizing forwarding across cores.

- **Load Balancer (LB) threads:** The main thread reads packets from the PCAP sequentially (I/O is inherently serial) and pushes each one to one of N LB queues.
- **LB → Fast Path routing = consistent hashing on the 5-tuple:** I compute `hash(5-tuple) % num_lbs` to pick which LB gets a packet, and then, inside that LB, `hash(5-tuple) % fps_per_lb` to pick which Fast Path (FP) worker thread actually processes it. I used MD5 over the packed struct of (src_ip, dst_ip, src_port, dst_port, protocol) as the hash function.
- **Why hash on the 5-tuple and not round-robin?** Because every packet belonging to the *same TCP/UDP flow* has to land on the *same worker thread*, every time — otherwise two different threads could race on the same flow's state (its SNI, whether it's already been marked blocked, its packet counter) and you'd get inconsistent results or need locks on every flow, which kills the whole point of parallelizing. Hashing the 5-tuple guarantees flow affinity: same flow → same hash → same thread, deterministically, with no coordination needed between threads.
- **Each FP thread owns its own flow table** (a plain Python dict, no locks needed) because no other thread ever touches those flows.
- **Output writer thread:** All FP threads push forwarded packets to one shared output queue; a single dedicated writer thread drains it and writes to the output PCAP — this avoids multiple threads fighting over one file handle.
- **Sentinel-based shutdown:** when the reader finishes, it pushes a `None` sentinel into every LB queue; each LB, on seeing the sentinel, forwards it to its own FP queues and exits; each FP, on seeing it, exits; finally a sentinel goes to the output queue so the writer flushes and closes. This is a clean way to signal "no more work" through a pipeline of queues without needing a shared shutdown flag with locks.
- **Stats aggregation:** a small `_Stats` class with one lock protects shared counters (total packets, per-app counts, per-thread dispatch counts) since those genuinely are touched by multiple threads.

Tech choices and why (be ready to justify each)

Choice

Why

Python, no Scapy/dpkt	Forced me to actually understand header layouts byte-by-byte instead of trusting a library; also makes the parsing logic transparent for the interview!
<code>struct</code> module	Standard library, precise control over binary layout (<code>!H</code> , <code>!I</code> etc. for big-endian network byte order)
Manual TLS ClientHello parsing	TLS's variable-length, TLV-style extension structure is a great example of a real-world binary protocol to hand-parse
<code>threading</code> + <code>queue.Queue</code> (not multiprocessing)	This is I/O/coordination-bound simulation work, not CPU-bound number crunching, and <code>queue.Queue</code> gives me a thread-safe, blocking producer-consumer channel for free
Consistent hashing (MD5 on 5-tuple)	Deterministic flow-to-thread affinity without shared locks per flow
Custom PCAP reader/writer	Understand the actual file format used by Wireshark/tcpdump rather than treating it as a black box

Known limitations (say these proactively — it shows maturity)

- **IPv4 only** — no IPv6 parsing yet.
- **No QUIC support** — QUIC (used by HTTP/3, and increasingly YouTube/Google traffic) runs over UDP and encrypts almost everything including what would be the SNI-equivalent, so this DPI approach — which depends on TCP port 443 and a visible TLS ClientHello — can't fingerprint QUIC flows today. That's actually the direction real-world DPI/CGNAT boxes are struggling with too.
- **No handling of Encrypted Client Hello (ECH)** — a newer TLS extension that encrypts the SNI itself; if that becomes widespread, SNI-based fingerprinting like mine stops working, and you'd have to fall back to traffic-pattern/ML-based fingerprinting (packet size/timing analysis).
- Rule engine is deliberately simple (IP/app/domain-substring) rather than a full policy language.

Likely counter-questions + how to answer

Q: Why not just use Scapy? A: Scapy is great for prototyping, but I wanted to actually understand the byte layout of these protocols — Ethernet, IPv4, TCP/UDP, TLS handshake — well enough to parse them by hand with `struct`. It also keeps the engine dependency-free and fast for this specific narrow use case.

Q: How do you handle a TCP segment where the TLS ClientHello is split across two packets (TCP segmentation)? A: Honestly, that's a real gap in the current version — I extract SNI from a single packet's payload, so a ClientHello split across TCP segments (which does happen if it's large, e.g. many cipher suites/extensions) would fail to parse. The fix would be TCP stream reassembly: buffer payload per flow using sequence numbers until you have a complete TLS record before attempting to parse it.

Q: Why hash to determine both LB and FP assignment — isn't that two levels of indirection for no reason? A: It mirrors how real hardware load balancers work — you often have front-end LBs distributing across racks/cores, and each of those doing a second-level dispatch to worker threads. It also let me test scaling both the number of LBs and FPs per LB independently as tunable parameters (`--lbs`, `--fps`).

Q: What's the time complexity of your rule check per packet? A: IP/app-type checks are $O(1)$ set lookups; domain-substring matching is $O(k)$ where k is the number of blocked domain patterns, since I do substring containment, not exact match. For a production system with many domain rules, I'd switch to a trie or Aho-Corasick for $O(\text{pattern length})$ multi-pattern matching.

Q: How would you test this? A: `generate_test_pcap.py` synthesizes traffic including TLS ClientHellos with known SNIs, HTTP requests, and DNS queries, and `tests/test_dpi.py` has unit/integration tests asserting the parser extracts the right fields and the engine's forward/drop decisions match the rules.

2. MeshPay (UPI Offline Mesh)

30-second pitch

"MeshPay is a FastAPI simulation of offline UPI payments — the idea is: what if you're a user in an area with no internet coverage, but you can still pay someone nearby over Bluetooth/local mesh, and the transaction settles automatically once *any* device in that mesh reaches a device with internet? I built the payment packet format, hybrid encryption for the payment payload, a gossip-protocol mesh simulator with virtual devices, and a settlement layer with idempotent duplicate protection and optimistic locking on account balances."

Why you built it

UPI in India needs internet on at least one side of the transaction. In low-connectivity areas — rural India, basements, crowded events — that's a real gap. The interesting systems problem isn't the UI, it's: how do you let a payment instruction propagate device-to-device without a live connection to the bank, guarantee it eventually reaches the backend exactly once (not zero

times, not twice), and keep it cryptographically safe the whole time it's bouncing between untrusted phones in the mesh?

Step-by-step: what happens on a payment

1. **User initiates a payment** (`POST /api/demo/send` with sender, receiver, amount, PIN)
→ `service/demo.py`:
 - The PIN is hashed with SHA-256 (never sent in plaintext — demo-only, not verified server-side, but shows the pattern).
 - A **nonce** (UUID) is attached specifically so that two identical payments (same sender/receiver/amount) still produce different plaintext, and therefore completely different ciphertexts — this matters for step 2.
 - A **signed_at** timestamp is attached so the server can later reject stale packets (replay/freshness window).
2. **Hybrid encryption of the payment payload** (`crypto/hybrid_crypto.py`) — this is the part I'd expect the deepest questions on:
 - The payload (sender, receiver, amount, pin_hash, nonce, signed_at) is JSON-serialized.
 - I generate a **fresh, random AES-256 key and a 12-byte IV for every single packet** — never reused.
 - The payload is encrypted with **AES-256-GCM**, which is an *authenticated* encryption mode — it produces ciphertext plus a 16-byte authentication tag. If even one byte of the ciphertext is altered anywhere in the mesh (by a malicious relay device, or corruption), decryption throws `InvalidTag` and the server refuses to process it. This is what stops a device in the mesh from tampering with the amount or receiver.
 - The AES key itself is then wrapped with **RSA-2048 + OAEP padding** using the server's public key. RSA can't encrypt the whole JSON payload directly (RSA-2048/OAEP can only encrypt ~245 bytes), so I only use RSA to encrypt the tiny 32-byte AES key — that's the textbook reason hybrid encryption exists at all.
 - Final wire format is just three things concatenated and base64-encoded: `[256-byte RSA-encrypted AES key][12-byte IV][AES-GCM ciphertext + tag]`.
 - Devices in the mesh only ever see this opaque base64 blob — they can relay it, but they can't read or forge it, because only the server holds the RSA private key.
3. **Injection into the mesh** — the encrypted packet (`MeshPacket`: packet_id, TTL=5, created_at, ciphertext) is handed to `phone-alice`, one of five simulated `VirtualDevices` (four phones with `has_internet=False`, one "bridge" phone with `has_internet=True`).

4. **Gossip protocol simulation** (`POST /api/mesh/gossip`) — mimics epidemic/gossip-style propagation used in real offline mesh networks (like Bluetooth mesh or delay-tolerant networks):
 - Every device broadcasts every packet it's currently holding (with TTL decremented by 1) to every other device.
 - Each device deduplicates by `packet_id` before storing — so a packet doesn't multiply infinitely inside one device's store.
 - TTL prevents a packet from bouncing forever if it never reaches a bridge device; once TTL hits 0 it stops propagating.
 - Call gossip repeatedly and you'll see the packet spread hop-by-hop until the bridge device (the one with internet) has it.
5. Bridge flush → server ingestion (`POST /api/mesh/flush`): the bridge device uploads every packet it's holding to the backend via `POST /api/bridge/ingest`, in parallel threads (one thread per bridge device, since in theory multiple bridges could exist).
6. **Server-side ingestion pipeline** (`service/bridge_ingestion.py`) — the ordered steps matter a lot here, and I did it deliberately in this order for efficiency and security:
 - **Step 1 — idempotency claim, BEFORE decryption**: I compute `SHA-256(raw ciphertext bytes)` as the idempotency key and try to atomically claim it in a dict guarded by a lock (`idempotency.claim()`), *before* spending any RSA/AES decryption cycles. If two different mesh paths deliver the same packet (very likely in a gossip network — that's the whole point of gossip, redundancy), the second one is dropped immediately as `DUPLICATE_DROPPED` without wasting CPU on decryption. This is the same pattern as `SETNX/putIfAbsent` — first writer wins, everyone else is instantly rejected.
 - **Step 2 — decrypt**: if it's genuinely new, decrypt it. If the GCM tag doesn't verify (tampering) or anything else fails, return `INVALID`.
 - **Step 3 — freshness check**: reject if `signed_at` is more than 24 hours old — protects against a very old packet resurfacing from a device that was offline for weeks and suddenly reconnects.
 - **Step 4 — settlement** (`service/settlement.py`): debit sender, credit receiver, write a ledger row — using **optimistic locking**: each account row has a `version` integer; the `UPDATE` statement includes `WHERE id = ? AND version = ?`, and if `rowcount == 0` (someone else modified that row between my read and write) it raises `OptimisticLockError` instead of silently overwriting a concurrent update. This matters because multiple bridge threads could be settling different transactions touching the same account concurrently.
 - Balance is checked against amount before debiting (`InsufficientFundsError`); both accounts must exist (`AccountNotFoundError`).

- Response outcome is one of **SETTLED** / **DUPLICATE_DROPPED** / **INVALID**, always keyed by the packet hash so the client can track exactly what happened to each packet it sent.
7. **Reset** (**POST** /**api/mesh/reset**) clears every device's packet store and the idempotency cache — useful for re-running the demo.

Tech choices and why

Choice	Why
FastAPI	Async-capable, automatic request validation via Pydantic models (MeshPacket , PaymentInstruction , SendRequest), good fit for a small REST demo with clear schemas
Pydantic schemas	Strong typing at the API boundary — a malformed packet gets rejected by FastAPI before it even reaches my logic
Hybrid RSA-OAEP + AES-256-GCM	RSA alone can't encrypt a payload this size; AES alone has no way to securely share a key with untrusted relay devices; combining them is the standard real-world pattern (same idea behind TLS itself)
AES- GCM specifically (not CBC)	Authenticated encryption — tamper-evidence is essential when the ciphertext physically passes through untrusted intermediate devices in the mesh
SHA-256 of ciphertext as idempotency key	Computed pre-decryption so duplicates are rejected cheaply; using the ciphertext (not a client-supplied ID) means a device can't fake a "new" packet ID to bypass the dedupe check — the actual bytes have to differ
In-memory SQLite (sqlite3 , :memory:)	Simple enough for a demo, but I kept the schema (accounts + transactions with version/packet_hash columns) production-shaped so swapping in PostgreSQL is a connection-string change, not a redesign
Optimistic locking (version column) vs. row-level locks	Avoids holding a database lock across the whole settlement transaction; better throughput when conflicts are actually rare, which they are in this simulation

<code>threading</code> for parallel bridge uploads	Same reasoning as DPI-engine — this is I/O-bound (network round trips to <code>/api/bridge/ingest</code>), so real threads are fine, no need for multiprocessing
TTL-bounded gossip	Prevents unbounded packet propagation storms in the simulated mesh

Known limitations (mention proactively)

- It's a **simulation** — "mesh" devices are in-process Python objects, not real Bluetooth/Wi-Fi Direct peers; a real implementation would need an actual local transport layer (BLE mesh, Wi-Fi Direct, or Nearby Connections API).
- PIN is hashed but never actually verified server-side — noted directly in the code comments; a real system would need proper authentication, not just a hash carried along for show.
- Single in-memory SQLite instance — not durable across restarts, not horizontally scalable; production would need PostgreSQL/Redis as noted in the code comments.
- No handling of partial network partitions where the bridge itself might be malicious (it can drop packets it's supposed to relay, though it can't forge or read them).

Likely counter-questions + how to answer

Q: Why hybrid encryption instead of just asymmetric (RSA) for everything? A: RSA-2048 with OAEP padding can only encrypt about 245 bytes of data — my JSON payload (sender, receiver, amount, hashes, nonce, timestamp) is bigger than that once it grows. Even if it weren't, RSA is orders of magnitude slower than AES for bulk data. So I use RSA only to encrypt a small, fresh AES key, and AES-GCM (fast, and authenticated) to encrypt the actual payload — that's literally how TLS itself bootstraps a session.

Q: What stops a malicious phone in the mesh from replaying an old valid packet to drain someone's account twice? A: Two independent protections: idempotency (same ciphertext hash can only settle once, checked before decryption even happens) and the 24-hour freshness window on `signed_at` inside the encrypted payload — so even a "new-looking" duplicate that changes the outer packet ID but replays the same ciphertext still gets caught by the ciphertext-hash check, and if somehow a genuinely re-signed old instruction showed up, the freshness check would catch it separately.

Q: Why not encrypt with the AES key only, and skip RSA entirely — just have devices agree on a shared secret? A: Because devices in the mesh are untrusted relays, not the payment endpoints — they should never be able to decrypt the payload, only pass it along. If I used a symmetric key, I'd have to somehow securely distribute that same key to every phone, which reintroduces the exact key-distribution problem asymmetric crypto solves. With RSA, only

the server's private key can unwrap the AES key — the phones carrying it never get to see the plaintext.

Q: Your account balance update uses optimistic locking — what happens on conflict, and why not just use a database transaction/row lock instead? A: On conflict (`rowcount == 0`), I raise `OptimisticLockError` and the ingestion returns `INVALID` for that attempt — in a production version, that packet would be retried by the bridge later. I chose optimistic locking over holding a row lock because in this scenario writes to the same account happening at the exact same instant are rare, so optimistic locking gives better concurrency; if this were a high-contention ledger (like a popular merchant account), I'd reconsider and go pessimistic, or move to a proper transactional DB with `SELECT ... FOR UPDATE`.

Q: What's the actual settlement — is it atomic? What if the process crashes between debiting the sender and crediting the receiver? A: Right now both updates happen inside one SQLite `get_db()` context manager that commits at the end and rolls back on any exception — so within this demo it's atomic at the SQLite-transaction level. In a distributed production system I'd want this to be a proper DB transaction (or a saga/outbox pattern if sender and receiver accounts lived in different services), so a crash mid-way can't leave money in limbo.

Q: How would you actually implement the mesh transport layer for real (not simulated)?

A: On Android, Google's Nearby Connections API or Wi-Fi Direct; more generally, BLE mesh profiles. The application-layer packet format and crypto I built wouldn't change — the "device.receive() / device.get_packets()" simulation layer is exactly the part that'd get replaced by a real radio transport.

How to connect the two projects in conversation (your bridge line)

If asked "why these two projects," use this: *"Both come from the same curiosity — what's actually happening at the packet level. DPI-Engine is about reading and classifying traffic that's already flowing on a real network. MeshPay is almost the inverse problem — building a small application-layer protocol and packet format from scratch, and reasoning about delivery guarantees (at-least-once relay, exactly-once settlement) when there's no reliable network at all. One taught me to parse and fingerprint protocols; the other taught me to design one, including the crypto and consistency guarantees around it."*

Quick-fire terms to have crisp definitions for

- **SNI (Server Name Indication)**: TLS extension in ClientHello telling the server which hostname the client wants, sent before the connection is encrypted — this is why DPI can still classify TLS traffic despite encryption.
- **5-tuple**: (src IP, dst IP, src port, dst port, protocol) — uniquely identifies a flow.
- **Consistent hashing (as used here)**: hashing the 5-tuple to deterministically route all packets of one flow to the same worker thread.
- **AEAD (Authenticated Encryption with Associated Data)**: encryption mode (like AES-GCM) that gives you confidentiality + tamper detection in one primitive.
- **Idempotency key**: a value derived from the request itself (here, hash of ciphertext) used to make retries/duplicates safe to process more than once.
- **Optimistic concurrency control**: assume conflicts are rare, detect them at write time via a version check, retry instead of locking upfront.
- **Gossip protocol**: nodes periodically exchange what they know with neighbors; information (or here, payment packets) spreads epidemically without central coordination.